Lecture 39: Deep Learning (Part-4) — Practice Problem Solutions

Complete mathematical derivations, architectural proofs, and code implementations for all 15 practice problems.

Solution 1: Inductive Bias and Deep Learning Taxonomy

- **FNN / MLP**: Assumes features are independent, scalar coordinates without intrinsic spatial ordering. It utilizes all-to-all dense connectivity ($\mathbf{W}\mathbf{x} + \mathbf{b}$), making it ideal for tabular matrices (CSVs) where column order can be arbitrarily permuted without altering semantic meaning.
- **CNN**: Imposes two powerful spatial inductive biases:
    1. *Local Connectivity*: Visual features are formed by spatially contiguous local patches.
    2. *Translation Equivariance / Weight Sharing*: Visual features (e.g. an edge) have the same mathematical meaning regardless of where they appear in the image.
- **RNN / LSTM**: Imposes a temporal inductive bias—sequential order matters, and current state depends dynamically on previous states via recurrent transitions: $\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t)$.

Solution 2: Digital Image Matrix Representation

1. **Dimensionality**:
    - Channels-Last (H, W, C): $(1080, 1920, 3)$
    - Channels-First (C, H, W): $(3, 1080, 1920)$
2. **Total Scalar Count**:

$$\text{Total Pixels} = 1080 \times 1920 \times 3 = \mathbf{6,220,800} \text{ scalar values}$$

3. **RGB Values**:
    - Pure Yellow (Red + Green): $(255, 255, 0)$
    - Pure Cyan (Green + Blue): $(0, 255, 255)$

- 50% Medium Gray: $(128, 128, 128)$

Solution 3: Mathematical Rationale for Pixel Normalization

1. **Vanishing / Exploding Gradients**: Raw pixel values up to 255 cause activations $z = \sum w_i x_i + b$ to become extremely large. In sigmoid/tanh layers, this pushes activations into saturation zones where derivatives are virtually zero ($\sigma'(z) \to 0$), freezing learning.
2. **Loss Surface Distortion**: If input magnitudes are in hundreds, gradient vectors $\nabla_{\mathbf{W}} L \propto \mathbf{x}$ become enormous, leading to violently oscillating updates and requiring an infinitesimally small learning rate.
3. **Min-Max Scaling**: Dividing by 255.0 scales all inputs into $[0.0, 1.0]$, creating an isotropic, well-conditioned loss bowl where standard learning rates ($\eta = 0.001$) converge smoothly.

Solution 4: Computer Vision Task Spectrum

Task	Question Answered	Output Format	Standard Model
1. Classification	What is this image?	Single scalar label $y \in \{1, \dots, C\}$	ResNet, VGG
2. Class + Loc	What and where is it?	Class label + 1 Bounding Box (x, y, w, h)	Custom CNN
3. Object Detection	Find and label all objects	List of tuples: $[(\text{Class}_k, x_k, y_k, w_k, h_k, \text{Conf}_k)]$	YOLO, Faster R-CNN
4. Semantic Segmentation	Classify every pixel	2D integer class mask of shape (H, W)	U-Net, DeepLabV3

Solution 5: Real-World Multi-Task Vision Architecture

Autonomous vehicles employ a unified multi-task backbone (such as a shared ResNet/RegNet feature extractor) coupled with task-specific heads:

1. **Semantic Segmentation Head**: Classifies driveable road surface vs sidewalks vs potholes pixel-by-pixel.

2. **Object Detection Head (3D Bounding Boxes)**: Predicts 3D boxes $[x, y, z, w, l, h, \theta]$ and velocities for surrounding vehicles, cyclists, and pedestrians.
3. **Lane Line Detection Head**: Predicts polynomial spline curves representing lane markers.
4. **Traffic Light & Sign Classifier**: Localizes signs and classifies current state (Red, Yellow, Green, Stop). All heads backpropagate into a shared visual representation in real time.

Solution 6: The Mathematical Catastrophe of Parameter Explosion

1. **Parameter Calculation**:
   - Input dimension $N_{\text{in}} = 1920 \times 1080 \times 3 = 6,220,800$
   - Output neurons $N_{\text{out}} = 256$

$$\text{Weights} = 6,220,800 \times 256 = 1,592,524,800$$

$$\text{Biases} = 256$$

$$\text{Total Parameters} = 1,592,524,800 + 256 = \mathbf{1,592,525,}$$

2. **Why it Fails**:
   - Storing 1.59 Billion float32 parameters requires ≈ 6.37 GB of RAM just for layer 1 weights (and > 19 GB with Adam optimizer states).
   - High variance: With 1.59B parameters, the network will instantaneously memorize training images, suffering catastrophic overfitting on unseen test data.

Solution 7: Spatial Locality and Translation Invariance

- **Spatial Locality**: In visual scenes, pixels close to each other form coherent semantic entities (edges, textures, surfaces). Flattening an image row-by-row into a 1D vector separates vertical neighbors by W elements, completely destroying 2D neighborhood relationships.
- **Translation Invariance**: An object's semantic label does not depend on its position. In an FNN, shifting a cat by 5 pixels redirects its activations to entirely distinct weights. In a CNN, sliding the exact same kernel across all positions guarantees identical feature extraction regardless of object location.

Solution 8: The Two-Stage CNN Blueprint

```
[ Input Image ]
       |
       v
[ Stage 1: Feature Extraction ] (Conv2d -> ReLU -> MaxPool2d blocks)
       |
       v  <--- 3D Tensor of Feature Maps (Channels x Height x Width)
[ Stage 2: Classification Head ] (Flatten -> Linear -> Dropout -> Softmax)
       |
       v
[ Output Class Probabilities ]
```

The data flowing between Stage 1 and Stage 2 is a **3D spatial feature tensor** (C, H, W) containing distilled spatial representations, which is flattened into a 1D vector before entering the classification head.

Solution 9: Hierarchical Visual Feature Learning

1. **Layer 1 (Low-Level)**: Kernels with small receptive fields (3×3) compute localized spatial derivatives, detecting oriented line segments, step edges, color contrasts, and corners.
2. **Layer 2–3 (Mid-Level)**: Subsequent convolutions combine low-level edge activations over larger effective receptive fields, detecting geometric primitives: curves, intersections, textures, circles, and surface patches.
3. **Deep Layers (High-Level)**: Final convolutions synthesize textures and parts into class-specific representations: wheels, eyes, animal ears, door handles, and complete object shapes.

Solution 10: Manual 2D Convolution Calculation

Given:

$$X = \begin{bmatrix} 2 & 0 & 1 & 3 \\ 1 & 2 & 0 & 2 \\ 0 & 1 & 3 & 1 \\ 2 & 1 & 0 & 2 \end{bmatrix}, \quad K = \begin{bmatrix} 1 & -1 \\ 2 & 0 \end{bmatrix}, \quad b = 1, \quad S = 1, \quad P = 0$$

1. **Output Dimensions**:

$$O = \left\lfloor \frac{W - K + 2P}{S} \right\rfloor + 1 = \frac{4 - 2 + 0}{1} + 1 = \mathbf{3} \implies \mathbf{3 \times 3}$$

2. **Top-Left Output Cell** $(0, 0)$: Receptive window in X: $\begin{bmatrix} 2 & 0 \\ 1 & 2 \end{bmatrix}$

$$\begin{aligned} y_{0,0} &= (2 \times 1) + (0 \times -1) + (1 \times 2) + (2 \times 0) + b \\ &= 2 + 0 + 2 + 0 + 1 = \mathbf{5} \end{aligned}$$

3. **Top-Right Output Cell** $(0, 2)$: Receptive window in X: $\begin{bmatrix} 1 & 3 \\ 0 & 2 \end{bmatrix}$

$$\begin{aligned} y_{0,2} &= (1 \times 1) + (3 \times -1) + (0 \times 2) + (2 \times 0) + b \\ &= 1 - 3 + 0 + 0 + 1 = \mathbf{-1} \end{aligned}$$

(Note: If followed immediately by ReLU, $\text{ReLU}(-1) = 0$*).*

Solution 11: Learnable Filters vs Classical Image Processing Filters

- **Classical Filters (Sobel, Prewitt, Laplacian)**: Hand-engineered with fixed, static mathematical constants derived by human researchers specifically for edge gradient calculations. They cannot adapt to task-specific patterns or complex textures.
- **CNN Learnable Kernels**: Initialized randomly (e.g. He or Xavier initialization). Their numerical weights are dynamically adjusted during training via backpropagation and gradient descent:

$$K_{m,n} \leftarrow K_{m,n} - \eta \frac{\partial \text{Loss}}{\partial K_{m,n}}$$

They autonomously discover the optimal mathematical feature extractors needed for the specific visual dataset.

Solution 12: The Fundamental Spatial Dimension Formula

Universal Formula:

$$O = \left\lfloor \frac{W - K + 2P}{S} \right\rfloor + 1$$

1. $W = 32, K = 5, P = 2, S = 1$:

$$O = \left\lfloor \frac{32 - 5 + 2(2)}{1} \right\rfloor + 1 = \frac{31}{1} + 1 = \mathbf{32} \implies \mathbf{32 \times 32}$$

2. $W = 224, K = 7, P = 3, S = 2$:

$$O = \left\lfloor \frac{224 - 7 + 2(3)}{2} \right\rfloor + 1 = \left\lfloor \frac{223}{2} \right\rfloor + 1 = 111 + 1 = \mathbf{112} \implies \mathbf{1}$$

3. **Proof of Same Padding**: We require $O = W$ when stride $S = 1$:

$$W = \frac{W - K + 2P}{1} + 1 \implies W - 1 = W - K + 2P \implies 2P =$$

Since K is odd, $K - 1$ is an even integer, guaranteeing P is an exact whole integer that preserves spatial dimension W perfectly. ■

Solution 13: Pooling Layer Theoretical Properties

1. 2×2 **Max Pooling Calculation**:

$$\text{Input: } \begin{bmatrix} 4 & 9 & 1 & 3 \\ 8 & 2 & 7 & 5 \\ 3 & 1 & 14 & 10 \\ 6 & 0 & 2 & 12 \end{bmatrix}$$

   - Top-Left Quad $[4, 9, 8, 2] \implies \max = \mathbf{9}$
   - Top-Right Quad $[1, 3, 7, 5] \implies \max = \mathbf{7}$
   - Bottom-Left Quad $[3, 1, 6, 0] \implies \max = \mathbf{6}$
   - Bottom-Right Quad $[14, 10, 2, 12] \implies \max = \mathbf{14}$

$$\text{Output Feature Map} = \begin{bmatrix} \mathbf{9} & \mathbf{7} \\ \mathbf{6} & \mathbf{14} \end{bmatrix}$$

2. **Zero Learnable Parameters Proof**: Max Pooling performs the non-linear selection $\max(a, b, c, d)$. Average pooling performs $\frac{1}{4}(a + b + c + d)$. Neither operation introduces a weight scalar w or bias scalar b. There are no gradient updates computed for the layer itself; gradients simply pass through to the maximizing input coordinate. Hence, $\text{Parameters} \equiv 0$.

3. **Translation Invariance**: If an edge within a 2×2 window shifts from $(0, 0)$ to $(0, 1)$, its value still represents the maximum of that quadrant. The pooled output value remains completely unchanged, conferring local translation invariance.

Solution 14: Flattening, Dense Reasoning, and Dropout

1. **Flattened Vector Length**:

$$N = C \times H \times W = 128 \times 7 \times 7 = \mathbf{6,272} \text{ nodes}$$

2. **Global Non-Linear Reasoning**: Convolutional layers capture only localized receptive fields. Dense layers connect every extracted feature to every class hypothesis, evaluating the global co-occurrence of parts across the entire image.
3. **Dropout Mechanics**: At each training iteration, each neuron is deactivated with probability $p = 0.5$. This prevents neurons from developing brittle co-dependencies (where neuron A only works if neuron B is active), forcing the network to learn redundant, robust feature representations.

Solution 15: End-to-End Architectural Parameter Audit

1. **Output Spatial Dimensions**:
   - Input: $(3, 32, 32)$
   - Layer 1 (`Conv2d` $K = 3, P = 1, S = 1$): $O = (32 - 3 + 2)/1 + 1 = 32 \implies \mathbf{(32, 32, 32)}$
   - Layer 2 (`MaxPool2d` $K = 2, S = 2$): $O = 32/2 = 16 \implies \mathbf{(32, 16, 16)}$
   - Layer 3 (`Conv2d` $K = 3, P = 1, S = 1$): $O = (16 - 3 + 2)/1 + 1 = 16 \implies \mathbf{(64, 16, 16)}$
   - Layer 4 (`MaxPool2d` $K = 2, S = 2$): $O = 16/2 = 8 \implies \mathbf{(64, 8, 8)}$
   - Layer 5 (`Flatten`): $64 \times 8 \times 8 = \mathbf{4,096}$ nodes
   - Layer 6 (`Linear`): $\mathbf{(128)}$
   - Layer 7 (`Linear`): $\mathbf{(10)}$
2. **Trainable Parameters**:
   - Layer 1: $(3 \times 3 \times 3 + 1) \times 32 = (27 + 1) \times 32 = \mathbf{896}$
   - Layer 2: $\mathbf{0}$
   - Layer 3: $(32 \times 3 \times 3 + 1) \times 64 = (288 + 1) \times 64 = \mathbf{18,496}$
   - Layer 4: $\mathbf{0}$
   - Layer 5: $\mathbf{0}$
   - Layer 6: $(4096 \times 128) + 128 = 524,288 + 128 = \mathbf{524,416}$

- Layer 7: $(128 \times 10) + 10 = 1280 + 10 = \mathbf{1,290}$

3. **Total Trainable Parameters**:

$$\text{Total} = 896 + 18,496 + 524,416 + 1,290 = \mathbf{545,098} \text{ Parameters}$$

(Notice that Layer 6 accounts for $524,416/545,098 \approx 96.2\%$ *of all parameters!)*